

**Investigación de APIs de Clima para la Plataforma WiseTrip:
Análisis Comparativo con Enfoque en Destinos de Latinoamérica**

Santiago Clavijo Suárez

Facultad de Ingeniería, Programa de Ingeniería de Sistemas

Proyecto de negocio WiseTrip

22 de agosto de 2026

Investigación de APIs de Clima para WiseTrip

Introducción

En WiseTrip, una de las partes más importantes del producto es mostrarle al usuario el clima del destino que está planeando, para que pueda organizar mejor su itinerario y recibir alertas si algo cambia. Para lograr esto se necesita conectar la plataforma con una API externa que nos pueda dar esa información, ya que no tiene sentido que nosotros mismos intentemos generar pronósticos del tiempo.

En este documento se muestra la investigación que hicimos sobre varias APIs de clima que existen en el mercado, comparándolas entre sí para justificar por qué elegimos Meteosource como la que vamos a usar en el proyecto.

Delimitación del proyecto

Antes de entrar en la comparación, quiero aclarar algo: en la presentación inicial mostramos a WiseTrip como una idea de alcance global, pero en la práctica el equipo decidió acotar varias cosas para que el proyecto fuera más manejable:

- La plataforma va a estar enfocada solo en destinos de Latinoamérica, no en el mundo entero. Por eso uno de los puntos que más nos importaba al comparar las APIs era qué tan buena es su cobertura en esta región.
- No vamos a manejar compra ni venta de tiquetes de avión. Lo que llamamos "reservas" en el modelo de negocio se refiere solo a hospedaje y actividades.
- Las actividades no son un listado genérico, sino una selección de cosas que se pueden hacer en la zona del destino (aventura, cultura, comida, etc.), pensadas según el clima esperado.

- Cuando hablamos de "transporte" nos referimos a la movilidad dentro del país o ciudad destino (buses, taxis, apps de movilidad), no al vuelo internacional del viajero.

Esto es relevante para la investigación porque cambia lo que realmente se necesita de una API de clima: nos interesa más que tenga buena cobertura y precisión en países de la región, que si es la API más completa o más usada a nivel mundial.

Qué buscábamos en una API de clima

Para poder comparar las opciones de forma un poco más objetiva, definimos algunos puntos que nos importaban, pensando en lo que WiseTrip necesita:

- Que tenga buena cobertura en países de Latinoamérica.
- Que tenga un plan gratuito que nos sirva mientras desarrollamos y probamos la plataforma.
- Que tenga pronóstico por horas, alertas de clima severo y algo de datos históricos, ya que esto lo necesitamos para armar el itinerario y las alertas.
- Que sea fácil de integrar y que su documentación (o al menos su interfaz) esté en español.

APIs revisadas

Meteosource

Meteosource usa modelos de machine learning para combinar datos de distintas fuentes meteorológicas y sacar un pronóstico único. Tiene un plan gratuito de 400 llamadas al día, y dos planes pagos bastante accesibles: Startup a USD 9 al mes (10 000 llamadas/día) y Standard a USD 29 al mes (20 000 llamadas/día), este último ya con pronóstico cada minuto, alertas de clima severo y datos astronómicos diarios. También se puede pedir historial de hasta 20 años como complemento pago.

Cubre 249 países, lo que incluye sin problema los destinos de Latinoamérica que nos interesan, y su interfaz viene en 15 idiomas, entre ellos español. Esto último nos ayuda bastante tanto para integrar la API como para mostrarle la información al usuario sin tener que traducir todo manualmente.

OpenWeatherMap

Es probablemente la API de clima más conocida. Su plan gratuito da cerca de 1 000 llamadas diarias e incluye clima actual, pronóstico a 5 días cada 3 horas y calidad del aire. Los planes pagos arrancan en unos USD 40 al mes, o se puede pagar por llamada con su producto One Call. El problema es que varias fuentes mencionan que la precisión baja en zonas rurales o en países con pocas estaciones meteorológicas, lo cual nos preocupa un poco pensando en destinos de Latinoamérica que no sean las capitales principales.

WeatherAPI.com

Ofrece clima en tiempo real, históricos, astronomía y alertas, con cobertura mundial. El inconveniente es que dejaron de tener un plan gratuito permanente: ahora solo dan una prueba de 14 a 30 días, y después toca pasarse a planes anuales cobrados por millón de solicitudes (cerca de EUR 990 al año), lo cual nos parece caro para un proyecto que apenas está empezando.

Open-Meteo

Es una API de código abierto que junta modelos de varios servicios meteorológicos nacionales (NOAA, DWD alemán, ECMWF europeo). No pide una API key y deja hacer hasta 10 000 llamadas al día gratis, con historial desde 1940 y bastante buena resolución espacial.

El detalle es que su plan gratuito es solo para uso no comercial. Como WiseTrip sí va a generar ingresos (comisiones, publicidad), tocaría contratar un plan comercial apenas el proyecto pase de prototipo a algo real con usuarios.

Weatherbit

Tiene un plan gratuito bastante limitado, pensado más para proyectos personales, y una prueba empresarial de 21 días con hasta 1 500 solicitudes diarias. Maneja historial de hasta 10 años en su línea agrícola. Es una alternativa que funciona, pero su documentación está en inglés y no vimos mucho enfoque en Latinoamérica.

Comparación entre las opciones

Para resumir todo lo anterior armamos la Tabla con los puntos que más nos importaban de cada proveedor.

Comparación de APIs

Proveedor	Cobertura en LATAM	Plan gratuito	Plan pago inicial	Datos históricos	Uso comercial en plan gratuito	Idioma / soporte
Meteosource	Sí, cobertura global (249 países) incluida LATAM	400 llamadas/día	USD 9/mes (10 000 llamadas/día)	20 años (add-on desde USD 0.005 por ubicación/día)	Sí, con mención y enlace visible	15 idiomas, incluido español
OpenWeatherMap	Sí, cobertura global; precisión menor en zonas rurales o con pocas estaciones	≈1 000 llamadas/día	USD 40/mes o pago por llamada (≈USD 0.0015/llamada tras el límite gratis)	Disponible en planes superiores	Sí	Múltiples idiomas

Proveedor	Cobertura en LATAM	Plan gratuito	Plan pago inicial	Datos históricos	Uso comercial en plan gratuito	Idioma / soporte
WeatherAPI.com	Sí, cobertura global	Ya no ofrece plan gratuito permanente (solo prueba de 14-30 días)	Desde planes anuales por millón de solicitudes (aprox. EUR 990/año)	Sí, incluida en planes pagos	Solo durante el periodo de prueba	Múltiples idiomas
Open-Meteo	Sí, cobertura global de alta resolución (hasta 1 km)	10 000 llamadas/día, sin necesidad de clave (API key)	Planes comerciales personalizados para mayor volumen y SLA	Desde 1940 (reanálisis)	No; el plan gratuito es exclusivamente para uso no comercial	Documentación en inglés; código abierto
Weatherbit	Sí, cobertura global	Plan gratuito limitado para uso personal/hobby	Prueba empresarial de 21 días, luego planes pagos según volumen	Hasta 10 años en datos agrícolas	Limitado en el plan gratuito	Documentación en inglés

Nota. Elaboración propia con base en la información publicada por cada proveedor

(Meteosource, 2026; OpenWeather, 2026; Open-Meteo, 2026; WeatherAPI.com, 2026; Weatherbit, 2026).

Por qué elegimos Meteosource

Con base en la comparación anterior, decidimos usar Meteosource como la API de clima para WiseTrip. Estas son las razones principales:

- Su cobertura de 249 países incluye todos los destinos de Latinoamérica que nos interesan por ahora.
- Los planes pagos son baratos (Aunque esta no sea una opción) comparados con las demás opciones, lo cual tiene sentido para un proyecto que apenas está arrancando.

- Deja usar los datos comercialmente incluso en el plan gratuito, solo pidiendo que se dé crédito visible al proveedor.
- Trae alertas de clima severo y pronóstico en intervalos cortos, que es justo lo que necesitamos para la parte de "actualizar información en tiempo real sobre clima" del modelo de negocio.
- Tiene soporte en español, lo que facilita tanto integrarla como mostrarle la información al usuario sin traducir todo a mano.

Conclusión

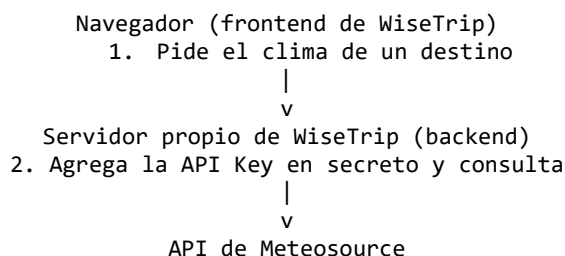
Elegir bien la API de clima es importante ya que de ahí depende buena parte de lo que WiseTrip promete: alertas útiles, un itinerario que realmente se adapte al clima del destino y una buena experiencia para el usuario. Después de comparar las cinco opciones según cobertura en Latinoamérica, costo, uso comercial y la facilidad de integración, Meteosource fue la que mejor se ajustó a lo que necesitamos ahora y a lo que probablemente vamos a necesitar más adelante.

Apéndice A: Guía Técnica de Implementación

En esta parte dejamos un boceto de código y los pasos que seguiríamos para conectar Meteosource con la página web de WiseTrip, para tenerlo de referencia cuando ya tengamos la plataforma construida.

Cómo se conecta todo

La idea no es llamar a Meteosource directamente desde el navegador del usuario, porque ahí quedaría expuesta nuestra API Key y cualquiera podría verla abriendo las herramientas de desarrollador del navegador. Por eso ponemos nuestro propio servidor en la mitad:



Así, el navegador solo habla con nuestro servidor, y es este el que conoce la clave y hace la petición real a Meteosource.

Pasos para hacerlo

- 1. Preparar el entorno.** Instalar Node.js 18 o superior y correr "npm install" para bajar las dependencias del servidor (express y dotenv).
- 2. Guardar la API Key como variable de entorno.** Crear un archivo ".env" (con base en un ".env.example") donde va la clave real. Este archivo no se sube al repositorio.
- 3. Armar el backend que hace de intermediario.** Un archivo server.js con rutas propias como "/api/buscar-lugar" y "/api/clima", que reciben la petición del navegador y por detrás llaman a Meteosource con la clave.
- 4. Buscar el place_id del destino.** Antes de pedir el clima, hay que conseguir el "place_id" del lugar llamando al endpoint "find_places_prefix" con el nombre que escribió el usuario.
- 5. Pedir el clima al endpoint /point.** Con el place_id (o lat/lon) se llama al endpoint "point" pidiendo las secciones "current", "hourly" y "daily", en español y en unidades métricas.
- 6. Mostrarlo en la página.** El código del navegador (app.js) recibe la respuesta de nuestro propio backend y la usa para pintar el clima actual y el pronóstico de los próximos días.

7. Llevarlo a la plataforma final. Cuando la página ya esté construida, hay que guardar el `place_id` de cada destino dentro del itinerario, cachear las respuestas para no gastar el límite diario de llamadas, y programar consultas periódicas para las alertas de clima severo.

Estructura del proyecto base

```
wisetrip-clima-demo/
├── server.js           Backend (Node.js + Express)
├── package.json
├── .env.example       Plantilla de variables de entorno
├── public/
│   ├── index.html     Página que ve el usuario
│   └── style.css
└── app.js             Lógica del navegador
```

Fragmentos de código

Variables de entorno

.env.example

```
# Copiar este archivo como ".env" y colocar la clave real.
METEOSOURCE_API_KEY=tu_api_key_aqui
METEOSOURCE_TIER=free

PORT=3000
```

Backend (server.js)

server.js (fragmento)

```
require('dotenv').config();
const express = require('express');
const app = express();
const API_KEY = process.env.METEOSOURCE_API_KEY;
const TIER = process.env.METEOSOURCE_TIER || 'free';
const BASE_URL = `https://www.meteosource.com/api/v1/${TIER}`;

app.use(express.static('public'));

// Ruta 1: buscar el place_id de un destino
app.get('/api/buscar-lugar', async (req, res) => {
  const texto = req.query.texto;
  const url = new URL(`${BASE_URL}/find_places_prefix`);
  url.searchParams.set('text', texto);
  url.searchParams.set('key', API_KEY);
  const respuesta = await fetch(url);
```

```

    const lugares = await respuesta.json();
    res.json(lugares);
  });

// Ruta 2: obtener el clima de un destino
app.get('/api/clima', async (req, res) => {
  const { place_id } = req.query;
  const url = new URL(`${BASE_URL}/point`);
  url.searchParams.set('place_id', place_id);
  url.searchParams.set('sections', 'current,hourly,daily');
  url.searchParams.set('language', 'es');
  url.searchParams.set('units', 'metric');
  url.searchParams.set('key', API_KEY);
  const respuesta = await fetch(url);
  const clima = await respuesta.json();
  res.json(clima);
});

app.listen(process.env.PORT || 3000);

```

Frontend (app.js)

app.js (fragmento)

```

// 1. Buscar el destino escrito por el usuario
async function buscarLugar(texto) {
  const respuesta = await fetch(
    `/api/buscar-lugar?texto=${encodeURIComponent(texto)}`
  );
  return respuesta.json();
}

// 2. Pedir el clima usando el place_id elegido
async function consultarClima(placeId, nombreVisible) {
  const respuesta = await fetch(

    `/api/clima?place_id=${encodeURIComponent(placeId)}`
  );
  const datos = await respuesta.json();
  pintarClimaActual(nombreVisible, datos.current);
  pintarPronostico(datos.daily);
}

```

Referencias

Meteosource. (2026). Pricing for weather API. <https://www.meteosource.com/pricing>

Meteosource. (2026). API documentation. Recuperado el 22 de agosto de 2026, de <https://www.meteosource.com/documentation>

Meteosource. (s.f.). Frequently asked questions. Recuperado el 22 de agosto de 2026, de <https://www.meteosource.com/faq>

Open-Meteo. (2026). Pricing. <https://open-meteo.com/en/pricing>

Open-Meteo. (s.f.). About. Recuperado el 22 de agosto de 2026, de <https://open-meteo.com/en/about>

OpenWeather. (2026). Self-service API pricing. <https://openweathermap.org/price>

WeatherAPI.com. (2026). Pricing. <https://www.weatherapi.com/pricing.aspx>

Weatherbit. (2026). Pricing. <https://www.weatherbit.io/pricing>